

Experiment – 3

Student Name: Naveen
Branch: CSE-AIML
Semester: 5th
Subject Name: ADBMS

UID: 24BAI70397
Section/Group: 24AIT_KRG-1_A
Date of Performance: 31/7/26
Subject Code: 24CSP-310

(A)

You are given with the Employee schema; the task is to find the highest score for each respective department.

For eg: department D1 have the highest score as 5.28. Hence in the output, final updated score for D1 should be 5.28.

Same with the other departments i.e., D2, D3 and D4.

OUTPUT:

EID	DEPT	SCORES	EID	DEPT	SCORES
1	D1	1	1	D1	5.28
2	D1	5.28	2	D1	5.28
3	D1	4	3	D1	5.28
4	D2	8	4	D2	8
5	D1	2.5	5	D1	5.28
6	D2	7	6	D2	8
7	D3	9	7	D3	9
8	D4	10.2	8	D4	10.2

Solution:

```
CREATE TABLE Employee
(
    EID INT PRIMARY KEY,
    DEPT VARCHAR(10),
    SCORES DECIMAL(5,2)
);

INSERT INTO Employee (EID, DEPT, SCORES)
VALUES
(1, 'D1', 1.00),
(2, 'D1', 5.28),
```

```
(3, 'D1', 4.00),  
(4, 'D2', 8.00),  
(5, 'D1', 2.50),  
(6, 'D2', 7.00),  
(7, 'D3', 9.00),  
(8, 'D4', 10.20);
```

```
UPDATE EMPLOYEE E1  
SET SCORES = (  
    SELECT MAX(SCORES)  
    FROM EMPLOYEE  
    WHERE DEPT = E1.DEPT  
);
```

```
SELECT * FROM EMPLOYEE;
```

(B)

A company maintains records of products sold by different salespersons. The management wants to prepare a sales performance report by identifying the salesperson(s) who generated the highest overall sales revenue. If multiple salespersons have the same highest total sales amount, all of them must be included in the final report. Return the seller ID(s) satisfying this condition.

Output

<u>seller_id</u>
3

Solution:

```
CREATE TABLE SalesPerson  
(  
    seller_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    city VARCHAR(50),  
    commission INT  
);
```

```
CREATE TABLE Company  
(  
    com_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    city VARCHAR(50)  
);
```

```
CREATE TABLE Orders  
(  
    order_id INT PRIMARY KEY,
```

```
order_date DATE,  
com_id INT,  
seller_id INT,  
amount INT,  
FOREIGN KEY (com_id)  
    REFERENCES Company(com_id),  
FOREIGN KEY (seller_id)  
    REFERENCES SalesPerson(seller_id)  
);
```

```
INSERT INTO SalesPerson  
VALUES  
(1, 'John', 'New York', 15),  
(2, 'Amy', 'Los Angeles', 13),  
(3, 'Mark', 'Chicago', 12),  
(4, 'Pam', 'Boston', 15);
```

```
INSERT INTO Company  
VALUES  
(1, 'RED', 'Boston'),  
(2, 'ORANGE', 'New York'),  
(3, 'YELLOW', 'Boston'),  
(4, 'GREEN', 'Austin');
```

```
INSERT INTO Orders  
VALUES  
(1, '2024-01-10', 1, 1, 1200),  
(2, '2024-01-12', 2, 1, 800),  
(3, '2024-01-15', 3, 2, 2500),  
(4, '2024-01-18', 1, 3, 1500),  
(5, '2024-01-22', 4, 2, 700),  
(6, '2024-01-25', 2, 3, 2000),  
(7, '2024-01-28', 3, 4, 3000),  
(8, '2024-01-30', 4, 4, 200);
```

```
SELECT SELLER_ID  
FROM ORDERS  
GROUP BY SELLER_ID  
HAVING SUM(AMOUNT) = (  
    SELECT MAX(TOTAL_SALES)  
    FROM (  
        SELECT SUM(AMOUNT) AS TOTAL_SALES  
        FROM ORDERS  
        GROUP BY SELLER_ID  
    )  
);
```

(C)

A company maintains employee salary records for different departments. The management wants to prepare a report containing employees whose salaries fall among the **top three highest distinct salary values** within their respective departments. If multiple employees receive the same qualifying salary, all such employees must be included in the final report. Display the department name, employee name, and salary for every qualifying employee.

Output

Department	Employee	Salary
IT	Max	90000
IT	Joe	85000
IT	Randy	85000
IT	Will	70000
Sales	Henry	80000
Sales	Sam	60000

Solution:

(a)

```
CREATE TABLE Department
(
    id INT PRIMARY KEY,
    name VARCHAR(50)
);
```

```
INSERT INTO Department (id, name)
VALUES
(1, 'IT'),
(2, 'Sales');
```

```
CREATE TABLE Employee
(
    id INT PRIMARY KEY,
    name VARCHAR(50),
    salary INT,
    departmentId INT,
    FOREIGN KEY (departmentId)
```

```
REFERENCES Department(id)
);
```

```
INSERT INTO Employee (id, name, salary, departmentId)
VALUES
(1, 'Joe', 85000, 1),
(2, 'Henry', 80000, 2),
(3, 'Sam', 60000, 2),
(4, 'Max', 90000, 1),
(5, 'Janet', 69000, 1),
(6, 'Randy', 85000, 1),
(7, 'Will', 70000, 1);
```

```
SELECT
    D.NAME AS DEPARTMENT,
    E.NAME AS EMPLOYEE,
    E.SALARY AS SALARY
FROM EMPLOYEE AS E
JOIN DEPARTMENT AS D
ON E.DEPARTMENTID = D.ID
WHERE
(
    SELECT COUNT(DISTINCT E2.SALARY)
    FROM EMPLOYEE E2
    WHERE E2.DEPARTMENTID = E.DEPARTMENTID
    AND E2.SALARY > E.SALARY
) < 3
ORDER BY D.NAME, E.SALARY DESC;
```